Academia\BackEnd\Academia Auth\Academia_Auth.c

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia - Authentication
*/

#include "Academia_Auth.h"
#include <stdio.h>
#include <string.h>
#include <time.h>

#define MAX_USERS 32

static char tokens[MAX_USERS][64];
static int ids[MAX_USERS];
static char roles[MAX_USERS][16];
static char names[MAX_USERS][64];
static int count = 0;

/* Copies text into a fixed array and always ends it with '\0'. */
static void copy(char *to, size_t size, const char *from) {
    if (size == 0) return;
    snprintf(to, size, "%s", from == NULL ? "" : from);
}

/* Wipes anything that would break the JSON reply. */
static void clean(const char *from, char *to, size_t size) {
    size_t i = 0;
    if (size == 0) return;
    while (from != NULL && from[i] != '\0' && i + 1 < size) {
        char c = from[i];
        to[i] = (c == '"' || c == '\\' || c == '\n' || c == '\r') ? ' ' : c;
        i++;
    }
    to[i] = '\0';
}

/* Position of a token in the list, or -1 if it is not there. */
static int find(const char *token) {
    int i;
    if (token == NULL || token[0] == '\0') return -1;
    for (i = 0; i < count; i++) {
        if (strcmp(tokens[i], token) == 0) return i;
    }
    return -1;
}

/* Sign in. The username, password and role must all match the same row. */
```

```c
int auth_login(sqlite3 *db, const char *username, const char *password,
               char *reply, size_t reply_size, char *token, size_t token_size,
               int *user_id, char *role, size_t role_size,
               char *name, size_t name_size) {
    sqlite3_stmt *row = NULL;
    const char *sql = "SELECT id, username, first_name, last_name, role FROM user "
                      "WHERE username = ? AND password = ? AND role = ?;";
    char full[64], safe_name[64], safe_role[16], safe_user[32];
    int id;

    if (db == NULL || username == NULL || password == NULL || role == NULL ||
        username[0] == '\0' || password[0] == '\0' || role[0] == '\0') {
        snprintf(reply, reply_size,
                 "{\"success\":false,\"message\":\"Enter a username, password and role.\"}");
        return 0;
    }

    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        snprintf(reply, reply_size, "{\"success\":false,\"message\":\"Database error.\"}");
        return 0;
    }
    sqlite3_bind_text(row, 1, username, -1, SQLITE_TRANSIENT);
    sqlite3_bind_text(row, 2, password, -1, SQLITE_TRANSIENT);
    sqlite3_bind_text(row, 3, role, -1, SQLITE_TRANSIENT);

    if (sqlite3_step(row) != SQLITE_ROW) {
        sqlite3_finalize(row);
        snprintf(reply, reply_size,
                 "{\"success\":false,\"message\":\"Invalid username, password or role.\"}");
        return 0;
    }

    id = sqlite3_column_int(row, 0);
    snprintf(full, sizeof(full), "%s %s", (const char *)sqlite3_column_text(row, 2),
             (const char *)sqlite3_column_text(row, 3));
    clean((const char *)sqlite3_column_text(row, 1), safe_user, sizeof(safe_user));
    copy(role, role_size, (const char *)sqlite3_column_text(row, 4));
    sqlite3_finalize(row);

    /* Forget the oldest session when the list is full. */
    if (count == MAX_USERS) {
        int i;
        for (i = 0; i < MAX_USERS - 1; i++) {
            strcpy(tokens[i], tokens[i + 1]);
            strcpy(roles[i], roles[i + 1]);
            strcpy(names[i], names[i + 1]);
            ids[i] = ids[i + 1];
        }
        count--;
    }
```

```

    snprintf(tokens[count], 64, "session-%d-%d", (int)time(NULL), id);
    ids[count] = id;
    copy(roles[count], 16, role);
    copy(names[count], 64, full);
    count++;

    copy(token, token_size, tokens[count - 1]);
    if (name_size > 0) copy(name, name_size, full);
    if (user_id != NULL) *user_id = id;

    clean(full, safe_name, sizeof(safe_name));
    clean(role, safe_role, sizeof(safe_role));
    printf("[Auth] %s signed in as %s\n", full, role);

    snprintf(reply, reply_size,
             "{\"success\":true,\"token\":\"%s\",\"user\":{\"id\":%d,\"name\":\"%s\","
             "\"username\":\"%s\",\"role\":\"%s\"}}",
             tokens[count - 1], id, safe_name, safe_user, safe_role);
    return 1;
}

/* Returns 1 only for a token that is in the list. */
int auth_check(const char *token, int *user_id, char *role, size_t role_size,
               char *name, size_t name_size) {
    int i = find(token);

    if (i < 0) {
        if (role_size > 0) role[0] = '\0';
        if (name_size > 0) name[0] = '\0';
        return 0;
    }
    if (user_id != NULL) *user_id = ids[i];
    copy(role, role_size, roles[i]);
    copy(name, name_size, names[i]);
    return 1;
}

/* Forget one session, or every session when the token is empty. */
void auth_logout(const char *token) {
    int i = find(token);

    if (token == NULL || token[0] == '\0') {
        count = 0;
        return;
    }
    if (i < 0) return;

    for (; i < count - 1; i++) {
        strcpy(tokens[i], tokens[i + 1]);
```

```
149         strcpy(roles[i], roles[i + 1]);
150         strcpy(names[i], names[i + 1]);
151         ids[i] = ids[i + 1];
152     }
153     count--;
154 }
```